Système & Scripting

Année Universitaire
2024-2025

Objectifs du module

- Maitriser les concepts de base liés à un scripts shell.
- Maitriser les différentes instructions utilisables dans un script shell.
- Ecrire des scripts simples d'exploitation Unix/Linux.
- Ecrire des scripts complexes d’administration en utilisant les boucles, menus et outils externes.
- Enrichir un script shell avec des fonctions et sous-shell.

Planification & Informations

- **Charge horaire** : 21h
- **Pré-requis** : Concepts de base liés aux systèmes d'exploitation Unix/Linux.
- **Mode d'évaluation** : 40% CC+60 % Examen .

Planification & Informations

- **Chapitre 1** : Introduction & Commandes de base
- **Chapitre 2** : Langage de programmation Shell
- **Chapitre 3** : Conditions & Boucles en Shell
- **Chapitre 4** : Sous-programmes en Shell

Système & Scripting

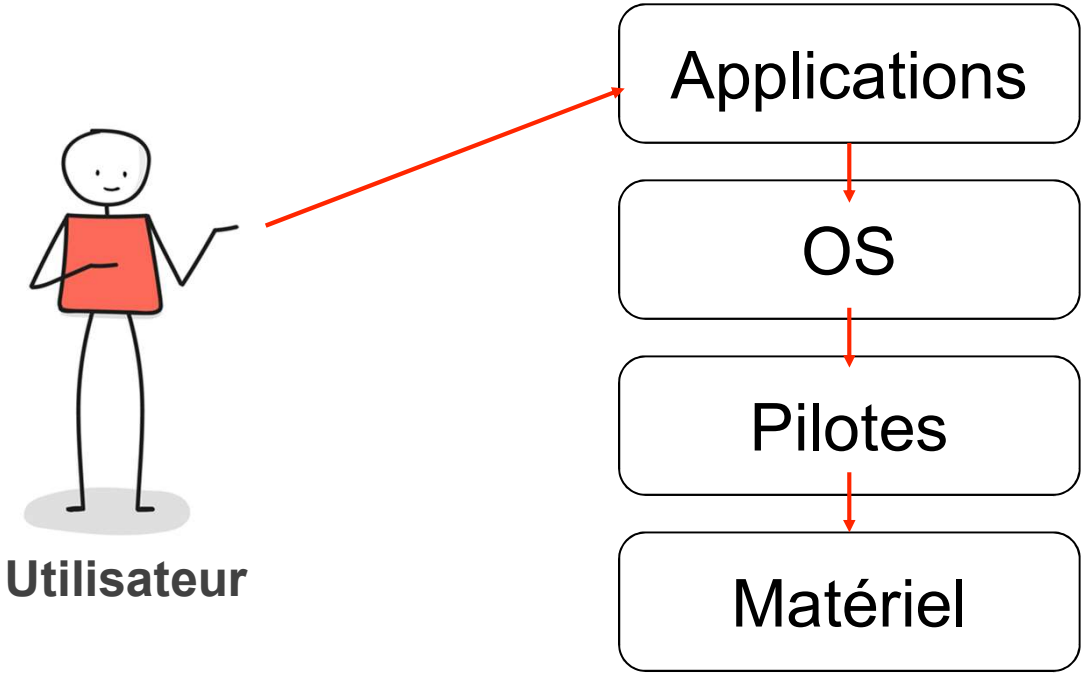
Chapitre I : Introduction & Commandes de base

Plan

1. Introduction aux systèmes d'exploitation Linux
2. Shell
3. Commandes Shell
4. Commandes de Base
5. Permissions et droits d'accès
6. Redirection & Pipes
7. Les Filtres

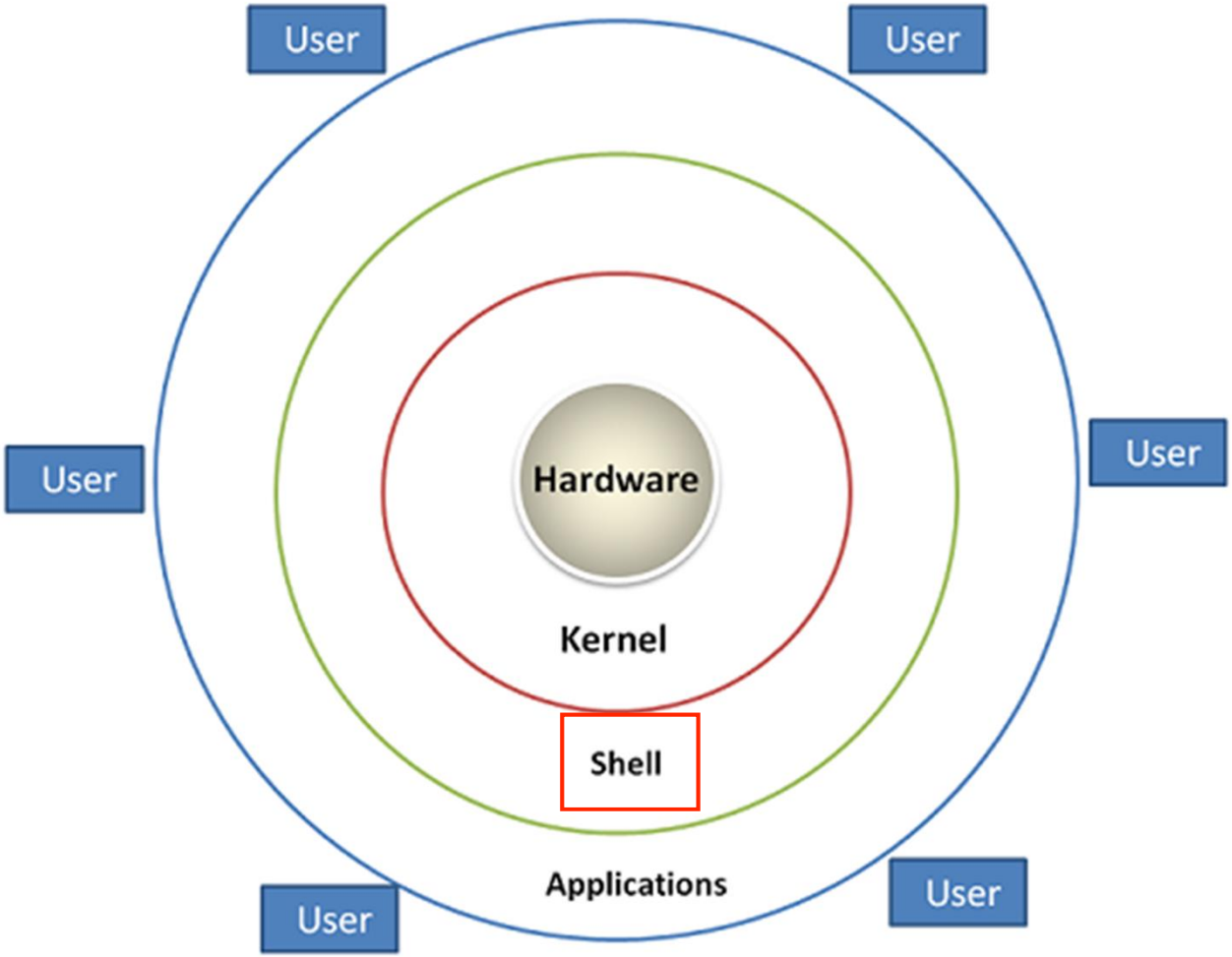
Introduction aux systèmes d'exploitation Linux

- Un **système d'exploitation**, ou **OS** (Operating System), définit un ensemble de programmes chargé d'établir une relation entre les différentes ressources matérielles, les applications et l'utilisateur.



Introduction aux systèmes d'exploitation Linux

Architecture Linux



Introduction aux systèmes d'exploitation Linux

Architecture Linux

- **Le noyau** : C'est la partie centrale du système d'exploitation qui gère les ressources du système et fournit des services aux programmes en cours d'exécution. Le noyau communique avec le matériel de l'ordinateur et gère les demandes d'accès aux ressources système telles que la mémoire et le processeur.

- **L'interpréteur de commande (shell)** : Cette composante fournit une interface pour que l'utilisateur puisse interagir avec le noyau du système d'exploitation par l'intermédiaire d'un langage de commandes, afin de permettre à l'utilisateur de piloter les périphériques.

- **Le système de fichiers** : Il s'agit d'une composante qui permet de gérer les fichiers et les dossiers sur le système dans une arborescence. Le système de fichiers fournit une interface utilisateur pour la gestion des fichiers, y compris la création, la suppression et la modification de fichiers et de dossiers

Introduction aux systèmes d'exploitation Linux

Rôle de l’OS

- Le composant principal d’un OS, c’est son Noyau ou Kernel en Anglais. il offre en générale les fonctionnalités suivantes:

- **Gestion du processeur :** Allocation du processeur entre les différents programmes grâce à un algorithme d’ordonnancement.

- **Gestion des processus :** Le système d'exploitation gère les processus (les programmes en cours d'exécution) sur l'ordinateur. Il alloue les ressources nécessaires à chaque processus et gère leur ordre d'exécution.

- **Gestion de la mémoire vive (RAM) :** Le système d'exploitation effectue la tâche d'allocation et de désallocation de l'espace mémoire aux programmes qui ont besoin de ces ressources.

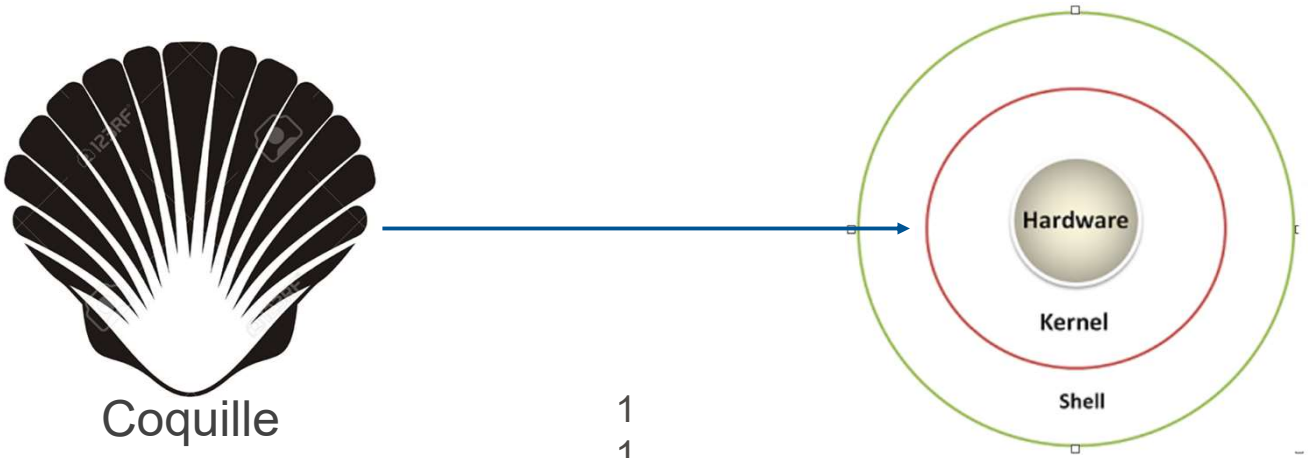
- **Gestion des entrées/sorties** : Le système d'exploitation gère les périphériques d'entrée/sortie (tels que les claviers, les souris, les imprimantes, les disques durs externes, etc.) connectés à l'ordinateur. Il permet aux programmes de communiquer avec les périphériques et gère les erreurs qui peuvent se produire.

- **Gestion des fichiers :** Le système d'exploitation gère les fichiers et les dossiers de l'ordinateur, permettant aux utilisateurs de stocker, de récupérer et de modifier des fichiers. Il s'occupe également de la sécurité et des autorisations d'accès aux fichiers.

Shell

Définition

- Shell est une « *coquille* » qui entoure le noyau Unix.
- Shell présente une interface entre l'utilisateur et le système d'exploitation.
- Ainsi, le shell est un programme qui permet à l'utilisateur de dialoguer avec le coeur du système (le noyau).

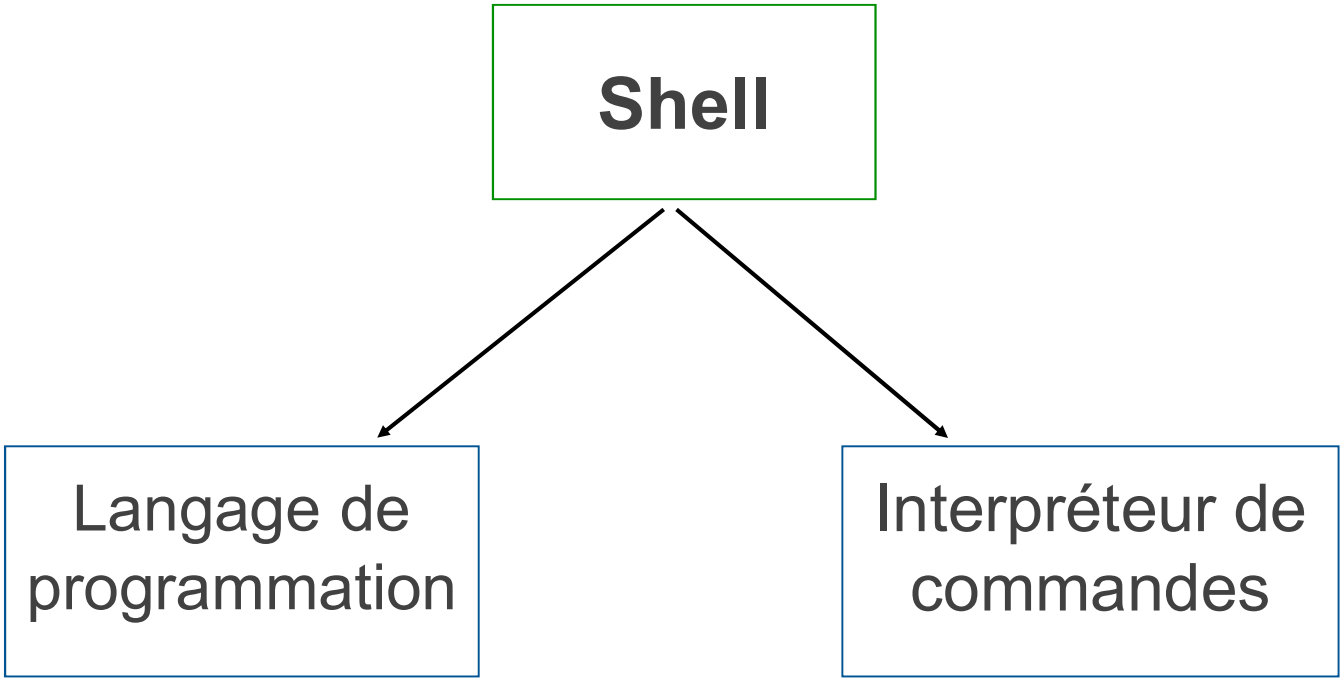


Shell

Rôle

Le shell possède un double rôle :

- Un interpréteur de commandes.
- Un langage de programmation script.



Shell

Rôle

- Shell interprète des commandes, des programmes ou des fichiers exécutables.
- Il assure :
  - Affichage de l'invite de commande ou prompt ($) d'attente de lecture au clavier.
  - Lecture d'une commande (validée par Return ou Entrée).
  - Analyse syntaxique (découpage en mot).
  - Interprétation des caractères spéciaux.
  - Exécution de la commande et retour au début.

Shell

Rôle

- Shell fournit à l'utilisateur un langage de programmation interprété.
- Il comporte :
  - les notions de variable, d’opérateur arithmétique, de structure de contrôle, de fonction, des opérateurs spécifiques (; |).

Shell

Différents Shell & Fonctionnalités

- Il existe plusieurs Shells, la commande « ***Cat /etc/shells*** » liste ces Shells
- Parmi les plus courants :
  - Bourne Shell (« /bin/sh ») : le Shell par défaut sous Linux
  - Korn Shell (« /bin/ksh ») pour lequel deux versions majeurs sont aujourd'hui couramment utilisées (ksh 88 et ksh 93) ;
  - cShell (« /bin/csh ») pour les utilisateurs préférant un langage apparenté au « C » ;
  - Job Shell (« /bin/jsh ») ;
  - tcShell amélioré (« /bin/tcsh ») améliorant le cShell tout en lui restant compatible ;

Shell

Différents Shell & Fonctionnalités

- Chaque utilisateur possède un shell par défaut, qui sera lancé à l'ouverture d'une invite de commande.
- Deux modes d'utilisation du Shell :
  - **Interactif** : l'utilisateur saisit et exécute ses lignes de commandes une par une dans un terminal ;
  - **Non interactif** : Shell interprète un ensemble de commandes à partir d'un fichier appelé shell script.

Commandes Shell

- Une ligne de commande est une chaîne de caractère constituée d'une commande ainsi que des arguments (paramètres) optionnels.
- Les commandes linux ont la syntaxe générale suivante :

Commande [Options] [Argument]

- ***Commande*** est le nom d'une commande interne ou un programme exécutable.
- ***Options*** **:** la plupart des commandes possèdent des options facultatives qui en modifient le comportement.
- ***Arguments/Paramètres*** sont les données d'entrées à la commande.

Commandes Shell

- Le shell distingue deux sortes de commandes : Commandes internes / Commandes externes.
  - **Commande interne** : Commande dont le code est implanté au sein de l'interpréteur de commande.
    - Exemples de commandes internes : cd , echo , pwd
  - **Commande externe** : Commande dont le code se trouve dans un fichier.
    - La localisation du code d'une commande externe doit être connue du shell pour qu'il puisse exécuter cette commande.

Commandes Shell

Utilitaires d’aide

- Il existe une multitude de commandes sous linux. Il est difficile de se rappeler de l’intégralité de ces dernières.
- Un ensemble d’utilitaires d’aide ont été mis à la disposition des utilisateurs pour faciliter la manipulation de cet environnement.

Commandes Shell

Utilitaires d'aide

Pages manuelles (Man)

- La commande « man » permet de fournir une page manuelle d'une commande envoyé en argument.

```
LS(1)                              User Commands                              LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort.

       Mandatory arguments to long options are  mandatory  for  short  options
       too.

       -a, --all
              do not ignore entries starting with .
```

Commandes Shell

Utilitaires d'aide

Pages infos (info)

- La commande « info » permet de fournir une page info d'une commande envoyé en argument.

```
File: coreutils.info,  Node: cp invocation,  Next: dd invocation,  Up: Basic op\
erations

11.1 `cp': Copy files and directories
=====================================

`cp' copies files (or, optionally, directories).  The copy is
completely independent of the original.  You can either copy one file to
another, or copy arbitrarily many files to a destination directory.
Synopses:

     cp [OPTION]... [-T] SOURCE DEST
     cp [OPTION]... SOURCE... DIRECTORY
     cp [OPTION]... -t DIRECTORY SOURCE...

   * If two file names are given, `cp' copies the first file to the
     second.
```

Commandes Shell

Utilitaires d'aide

Commande (apropos)

- La commande « apropos » est une commande POSIX qui permet de lister les manuels dont la description comprend les mots passés en arguments.

```
[esprit@localhost ~]$ apropos copy
bcopy                  (3)  - copy byte sequence
bcopy                  (3p)  - memory operations (LEGACY)
bcopy [bstring]        (3)  - byte string operations
copysign               (3)  - copy sign of a number
copysign               (3p)  - number manipulation function
copysignf [copysign]   (3)  - copy sign of a number
copysignf [copysign]   (3p)  - number manipulation function
copysignl [copysign]   (3)  - copy sign of a number
copysignl [copysign]   (3p)  - number manipulation function
cp                     (1)  - copy files and directories
cp                     (1p)  - copy files
```

Commandes Shell

Utilitaires divers

- ***date*** : affiche la date,
- ***cal*** : affiche le calendrier,
- ***who*** : affiche les utilisateurs connectés,
- ***whoami*** : affiche votre nom d'utilisateur,
- ***uname -a*** : affiche toutes les informations sur le système,
- ***whatis*** [commande] : affiche une description de la commande,
- ***hostname***: afficher le nom de la machine,
- ***halt ou shutdown -h now ou init 0***: éteindre l'ordinateur,
- ***reboot ou shutdown -r now ou init 6:*** rebooter l'ordinateur,
- ***lsb_release –d*** : afficher le nom de la distribution.
- **type [commande]** : Retourne le chemin absolu d'une commande externe ou indique s'il s'agit d'une commande interne (shell builtin).
- **which [commande]** : Recherche les exécutables des commandes selon les chemins indiqués dans la variable PATH et retourne le chemin absolu de la commande externe. Indique qu'il n'y a pas de chemin pour les commandes internes.
- **whereis [commande]** : Fonctionne comme which, mais affiche aussi le chemin vers les pages manuelles des commandes**.**

Commandes Shell

Chemin absolu vs chemin relatif

Chemin Absolu

Un chemin absolu décrit l'emplacement d'un fichier ou d'un répertoire en partant de la racine du système de fichiers.

Il commence toujours par / : le répertoire racine.

Exemples : /home/user/Téléchargements

Chemin Relatif

Un chemin relatif décrit l'emplacement d'un fichier ou d'un répertoire par rapport à votre répertoire courant.

Il ne commence jamais par /

Exemples : documents (dans le répertoire courant)

../documents (dans le répertoire parent)

./scripts (dans un sous-répertoire du répertoire courant)

Commandes de base

Commande pwd

- La commande « pwd » permet d'afficher le chemin absolu du répertoire courant

Commande (cd)

- La commande «cd » permet de se déplacer dans l'arborescence. Sans paramètre, elle nous renvoie vers le répertoire personnel de l'utilisateur actuel, sinon au chemin spécifié en argument,
- Avec les arguments listés ci-dessous, elle permet de :
  - **.** : se déplacer dans le répertoire courant
  - **..** : se déplacer vers le répertoire parent
  - **-** : se déplacer vers le répertoire précédent
  - **/** : se déplacer vers le répertoire racine
  - **~** : se déplacer vers le répertoire personnelle de l'utilisateur courant

Commandes de base

.

Commande ls

- Cette commande permet principalement de lister le contenu d'un répertoire. Nous pouvons lui associer plusieurs options dont les plus évidentes sont :
  - **-l** : Lister les métadonnées relatives à un fichier ou un répertoire.
  - **-a** : Afficher tous les fichiers, y compris les fichiers cachés.
  - **-R** : Pour assurer un affichage récursif (le contenu du répertoire et de ses sous répertoires).
  - **-i** : Afficher le numéro d'i-node des fichiers et des répertoires.
  - -**d** : Afficher les informations d'un dossier.

Commandes de base

Manipulation des fichiers

Commande touch

La commande « touch » permet de créer un ou plusieurs fichiers vides.

Commande gedit

La commande gedit est un éditeur de texte graphique.

Commande nano

La commande « nano » est une commande d'édition de fichier.

Commande vi

La commande « vi » est une commande d'édition de fichier.

Commande cat

La commande « cat » permet de:

- **cat** *fichier*: Afficher le contenu du *fichier*
- **cat** *fichier1 fichier2***:** Afficher le contenu des fichiers *fichier1* et *fichier2* concaténés.
- **cat -n** *fichier*: Afficher le contenu du *fichier* avec des numéros de ligne

Commande nl

nl *fichier*: Afficher le contenu du *fichier* avec des numéros de ligne (sans tenir en compte les lignes vides.

Commandes de base

Manipulation des fichiers

Commande more

La commande more permet d'afficher le contenu d'un fichier texte page par page, ce qui est particulièrement utile pour lire des fichiers longs dans le terminal

Commande head

La commande La commande « head » permet d'afficher par défaut les dix premières lignes.

- **head -n** nbre *fichier*: afficher les nbre premières lignes du fichier.
- **head -c** nbre *fichier*: afficher les nbre premiers caractères du fichier.

Commande tail

La commande « tail » permet d'afficher par défaut les dix dernières lignes.

- **tail -n** nbre *fichier*: afficher les nbre dernières lignes du fichier
- Tail +n nbre fichier: afficher les dernières ligne à partir de la ligne numéro nbre.
- **tail -c** nbre *fichier*: afficher les nbre derniers caractères du fichier

Commande tr

La commande « tr » permet de remplacer une chaine par une autre.

tr *'ancienne chaîne' 'nouvelle chaine'*

Commandes de base

Manipulation des fichiers

Commande wc

La commande « wc » permet d'afficher le nombre de lignes, nombre des mots, nombre d'octets.

- ***wc -l*** *fichier*: Afficher le nombre de lignes du *fichier*.
- **wc -w** *fichier*: Afficher le nombre de mots du *fichier*.
- **wc -c** *fichier*: Afficher le nombre d'octets du fichier.
- **wc -m** *fichier*: Afficher le nombre de caractères du fichier.

Commande split

La commande « split» permet de fragmenter un fichier. Chaque fragment est stocké dans des fichiers PREFIXaa, PREFIXab …

- *split -b* size *fichier partie*: Fragmente fichier en des fichiers de taille size (octets) nommés partieaa, partieab, etc.
- *split -l* nb *fichier partie*: Fragmente fichier en des fichiers de taille nb (lignes) nommés partieaa, partieab, partieac…

Commandes de base

Manipulation des fichiers

Commande paste

La commande paste sous Linux est utilisée pour fusionner les lignes de plusieurs fichiers ou l'entrée standard. Elle place les lignes côte à côte avec un délimiteur spécifié (par défaut une tabulation).

- paste –d «délimiteur» fichier1 fichier2 : concaténe horizontalement chaque ligne du fichier1 avec la ligne correspondante du fichier2 séparé par « délimiteur » .
- paste –s fichier1 fichier2 : Fusionner tout le contenu de chaque fichier en une seule ligne

Commande join

La commande « join » permet du fusionner les lignes de deux fichiers ayant des champs communs.

Join [options] fichier1 fichier2

Les fichiers doivent être triés par la clé de jointure. Utilisez sort si nécessaire

Commandes de base

Manipulation des répertoires

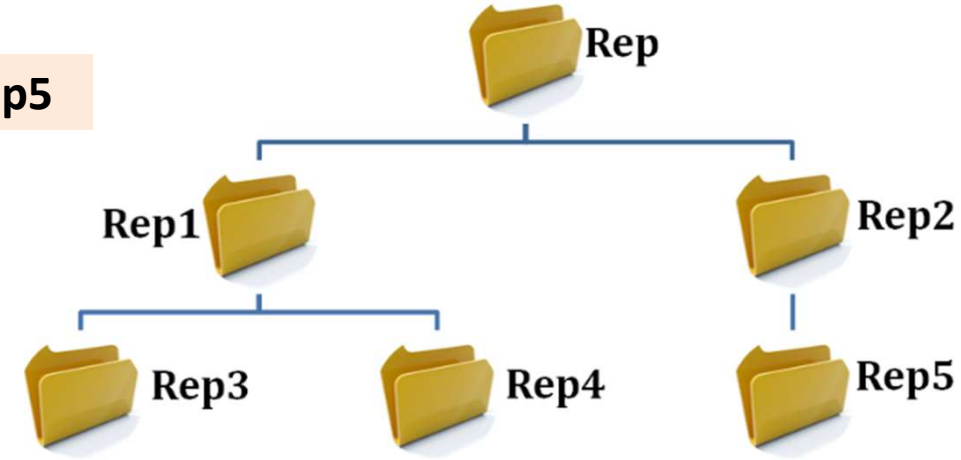
Commande mkdir

La commande « mkdir » permet la création d'un répertoire sous le chemin spécifié en paramètres.

Pour la création d'une arborescence, il faut ajouter l'option « -p ».

Exemple

```
$ mkdir -p  Rep/Rep1/{Rep3,Rep4} Rep/Rep2/Rep5
```



Commande rmdir

La commande « rmdir » permet de supprimer un répertoire vide.

Commandes de base

Manipulation des fichiers et des répertoires

Commande cp

La commande « cp » permet de copier une entité (fichier/répertoire) d'un emplacement à un autre.

- **-i:** interactif: demande l'autorisation d'écraser.
- **-r:** récursif: Copie les sous-répertoires et leur contenu.

Commande mv

La commande « mv » permet de renommer ou déplacer un fichier ou un répertoire, d'un répertoire à un autre.

Commande rm

Cette commande permet de supprimer une entité (fichier/répertoire).

- **-i:** interactif: demande l'autorisation d'écraser.
- **-r:** récursif: supprime les sous-répertoires et leur contenu.

Permissions et droits d'Accès

Commande chmod

La commande « chmod » permet de changer les permissions d'accès d'un fichier ou d'un répertoire.

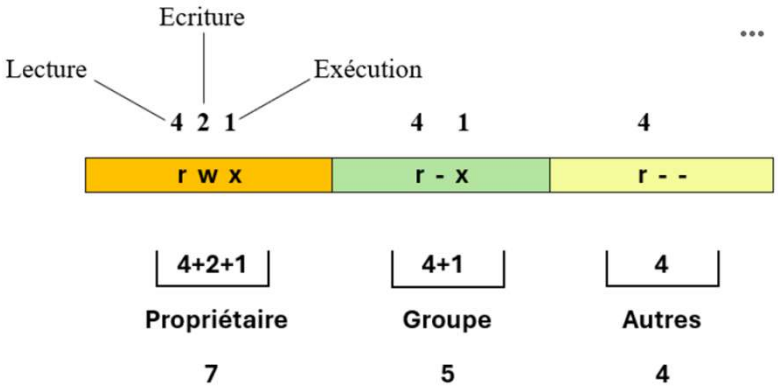
Méthode latérale

chmod [u g o a] [+ - =] [r w x] nom_du_fichier/répertoire

Méthode octale

Chaque chiffre correspond dans l'ordre à l'utilisateur, le groupe et le reste du monde

- A chaque droit correspond une valeur : (r → 4, w → 2, x → 1)
- On utilise la somme des droits que l'on veut affecter (0 signifie aucun droit)



chmod 754 nom_du_fichier/répertoire

Permissions et droits d'accès

Commande umask

La commande umask permet de définir les droits par défaut d'un fichier ou répertoire à sa création. Autrement dit elle indique les droits qu'il faut retirer lors de leur création.

```
Nouveaux droits par défaut (Répertoire)= 777 (en binaire) AND Not umask (binaire)
Nouveaux droits par défaut (fichier)=666 (en binaire) AND Not umask (binaire)
```

Exemple:

Calculer les nouvelles droits par défaut pour les fichiers et les répertoires en appliquant le umask 222:

- 222 =010 010 010

Répertoire

111 111 111 AND
Not (010 010 010)

111	111	111
101	101	101
101	101	101

→d r-x r-x r-x

Fichier

110 110 110 AND
Not (010 010 010)

110	110	110
101	101	101
100	100	100

→- r– r– r–

Redirection & Pipes

Redirection d'entrées & sorties

Lorsque vous exécutez une commande Linux, il y a trois flux de données qui jouent un rôle dans cette commande :

▪ **L'entrée standard (stdin)**: est la source des données d'entrée. Par défaut, stdin est tout texte saisi au clavier. Son ID de flux est 0.

▪ **La sortie standard (stdout)**: est le résultat de la commande. Par défaut, elle est affichée à l'écran. Son ID de flux est 1.

▪ **L'erreur standard (stderr)**: est le message d'erreur (le cas échéant) produit par les commandes. Par défaut, stderr est également affiché à l'écran. Son ID de flux est 2.

Périphérique	Association par défaut	Descripteur de fichier
Standard input buffer	Clavier	0
Standard output	Ecran	1
Standard output error	Ecran	2

Redirection & Pipes

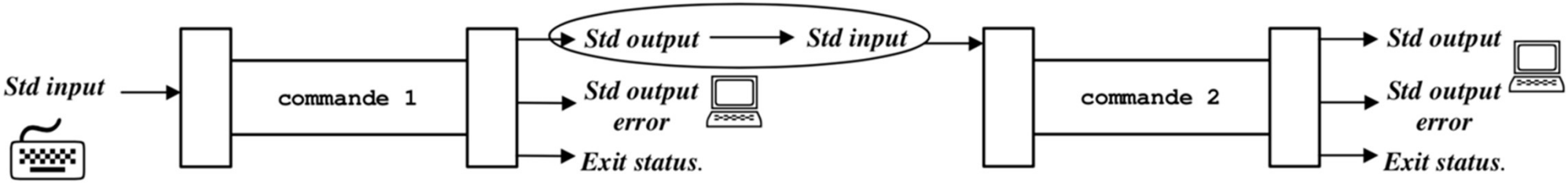
Redirection d'entrées & sorties

Syntaxe	Description ...
Cmd > file **ou** **Cmd 1>file**	Place le résultat de la sortie standard (stdout) de « Cmd » dans le fichier nommé file. Si file existe, alors il est écrasé et recréé; sinon, il est créé tout simplement.
Cmd 2> file	Les messages d'erreur issus de la commande « Cmd » sont redirigés vers le fichier file
Cmd >> file	ajoute à la fin du fichier file le résultat de la sortie de « Cmd » (stdout). Si le chier n'existe pas, il est créé.
Cmd 2>> file	comme ci-dessus, mais concerne la sortie d'erreur (Stderr)
Cmd > file 2>&1 **Ou** **Cmd >>file 2>&1**	Redirige le flux d'erreurs et la sortie standard de « Cmd » à file
Cmd < file	Prend comme entrée (stdin) de la commande « Cmd » le contenu du fichier file.
Cmd << mot_clé	Lire progressivement depuis le clavier jusqu'à la saisie de la chaine "mot_clé"

Redirection & Pipes

Redirection d'entrées & sorties

- Le tube (ou pipe en anglais) est un mécanisme permettant la communication entre plusieurs commandes.
- Dans un pipe, la sortie de la première commande devient l'entrée de la seconde.
- le shell prend à sa charge la connexion de la sortie standard de la première commande sur l'entrée standard de la deuxième.



Les Filtres

Commande find

La commande **find** est utilisée pour rechercher des fichiers et des répertoires dans un système de fichiers en fonction de divers critères.

find [rep] [options] [expression]

-name	Recherche par **nom** de fichier. ...
-type	Recherche par **type** de fichier.
-user	Recherche par **propriétaire**.
-group	Recherche par **appartenance** à un **groupe**.
-size	Recherche par taille de fichier.
-atime	Recherche par date de **dernier accès**.
-mtime	Recherche par date de **dernière modification**.
-ctime	Recherche par date de **création**.
-perm	Recherche par **autorisation d'accès**.

Les Filtres

Caractères spéciaux (Jokers)

- * une chaîne quelconque (même vide)
- ? un caractère quelconque
- [...] un caractère quelconque appartenant à l'ensemble « ... »
- [!...] un caractère quelconque hors de l'ensemble

Exemples :

- **cp *.c bakup** : copie de tous les fichiers ayant le suffixe .c sous le répertoire bakup
- **ls *.[csp]** : lister tous les fichiers ayant l'un des suffixes .c, .s ou .p
- **ls /usr/[a-z][0-9]** : lister tous les éléments de /usr dont le nom est composé de deux caractères : une minuscule suivie d'un chiffre
- **ls /bin/[!a-z]*** : lister tous les éléments de /bin dont le nom ne commence pas par une minuscule.

Les Filtres

Exécution de commande find avec -exec

L'option -exec de la commande find permet d'exécuter une commande sur chaque fichier ou répertoire trouvé par find.

find [chemin] [options] -exec [commande] {} \;

[chemin] : Le répertoire à partir duquel la commande find commencera la recherche.
[options] : Les critères de recherche : -name, -type, -perm .....
[commande] : La commande que vous souhaitez exécuter sur chaque fichier ou répertoire trouvé.
{} : Un espace réservé pour le nom du fichier/ répertoire trouvé. find remplace {} par le nom du fichier/répertoire actuel.
; : Termine la commande -exec.

Exemple: *Supprimer tous les fichiers d'extension .csv dans le répertoire rep2*

```
esprit@esprit:~$ ls rep2
fichier1  file1_sorted.csv  file3            info.txt  repertoire1  ...
fichier2  file1.txt         file4            lien_s
file      file2             file_sorted.csv  num.txt
esprit@esprit:~$ find rep2 -type f -name "*.csv" -exec rm {} \;
esprit@esprit:~$ ls rep2
fichier1  file       file2  file4     lien_s   repertoire1
fichier2  file1.txt  file3  info.txt  num.txt
```

Les Filtres

Commande grep

La commande **grep** est utilisé pour rechercher des motifs (patterns) spécifiques dans des fichiers ou des entrées standard .

grep [options] [motifs] fichier

Option	*Description*
-c	Compte le nombre de lignes qui contiennent le mot clef recherché.
-r	Effectue une recherche récursive dans les répertoires.
-i	Ignore la casse lors de la recherche.
-n	Affiche le numéro de ligne de chaque correspondance.
-v	Inverse la recherche pour afficher les lignes qui ne correspondent pas au motif.

Exemple: *Chercher dans le fichier /etc/passwd les lignes commençant par la lettre a en affichant leur numéro (numéro de ligne)*

```
esprit@esprit:~$ grep -n "^a" /etc/passwd
29:avahi-autoipd:x:110:119:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
33:avahi:x:114:121:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin
```

Les Filtres

Commande Sort

La commande sort est utilisée pour trier les lignes d'un fichier ou d'une entrée standard.

sort [options] file

Option	Description
-n	*Tri numérique*
-r	*Tri en ordre inverse (décroissant)*
-t	*Spécifie un délimiteur pour les champs*
-k	*Tri par une colonne spécifique*
-b	*ignore les espaces placés en début de champ*
-f	*Aucun différentiation n'est faite entre minuscules et majuscules*
-d	*Tri de type dictionnaire*
-M	*trier chronologiquement les mois*
-u	*Supprime les lignes dupliquées*

Exemple: trier le fichier info.txt selon l'âge sachant que la structure de fichier est comme suit:
Nom:Prénom:Age

```
esprit@esprit:~/Desktop$ cat info.txt
Ben Ali:Mohamed:23
Ben Salah:Nour:16
Naimi:Farid:21
trabelsi:Salim:14
Aloui:Mariem:9
esprit@esprit:~/Desktop$ sort -t: -k3n info.txt
Aloui:Mariem:9
trabelsi:Salim:14
Ben Salah:Nour:16
Naimi:Farid:21
Ben Ali:Mohamed:23
```

Les Filtres

Commande cut

La commande **cut** permet d'extraire des portions de texte à partir de fichiers ou d'une entrée standard.

cut [options] file

Options	Rôle
-f (fields)	Sélectionner des **champs** spécifiques, basés sur le délimiteur
-d (delimiter)	Spécifier un délimiteur pour les champs (par défaut, le tabulateur est utilisé)
-c (characters)	Sélectionner des plages de **caractères**

Exemple: Afficher seulement les N° de Facture et les noms des clients du fichier Suivi_Clients.csv.

```
esprit@esprit:~/Downloads$ cat Suivi_Clients.csv
N°Facture: Date Facture: Nom  Client: Montant: Etat: réglé le: Mode de paiement
FA-2023-11: 21/11/2023: AvatarTech: 480Dt: Réglé: 05/01/2024: espèce
FA-2023-08: 25/08/2023: InnovTech: 1400Dt: Réglé: 03/12/2023: chèque
FA-2023-07: 15/07/2023: Vivalto Clinic: 3750Dt: Réglé: 21/09/2023: chèque
FA-2023-09: 28/09/2023: ClickBank: 2780Dt: Réglé: 03/12/2023: virement
FA-2023-04: 26/06/2023: YorkBank: 2480Dt: Réglé: 03/08/2023: virement
FA-2024-01: 14/01/2024: Idea Hotel Tunisia: 3880Dt: Réglé: 13/03/2024 chèqu
FA-2024-02: 27/02/2024: Association SOS orphelin: 1270DT: Réglé: 29/04/2024
```

```
esprit@esprit:~/Downloads$ cut -d : -f 1,3 Suivi_Clients.csv
N°Facture: Nom  Client
FA-2023-11: AvatarTech
FA-2023-08: InnovTech
FA-2023-07: Vivalto Clinic
FA-2023-09: ClickBank
FA-2023-04: YorkBank
FA-2024-01: Idea Hotel Tunisia
FA-2024-02: Association SOS orphelin
```

Les Filtres

Commande awk

La commande **awk** est un outil polyvalent utilisé pour traiter et analyser du texte dans les systèmes Unix/Linux. Elle est capable de lire des fichiers ligne par ligne, de diviser chaque ligne en champs, et d'effectuer diverses opérations sur ces champs.

awk 'programme' fichier

- **programme** : Contient les instructions pour traiter les données.
- **fichier** : Le fichier ou les données à analyser.

Un programme awk est constitué de modèles et actions :

pattern { action }

pattern : Spécifie les lignes sur lesquelles l'action doit être effectuée.

action : Décrit ce qui doit être fait avec les lignes correspondantes.

$1, $2, $3, ... : Représentent les champs de la ligne (colonnes). $0 : Représente la ligne entière.

-Par défaut, les champs sont séparés par des espaces ou des tabulations. Vous pouvez spécifier un délimiteur différent avec -F.

Les Filtres

Commande awk

Exemples:

1- Afficher le champ n°1 du fichier Suivi_Clients.csv.

```
esprit@esprit:~/Downloads$ awk '{ print $1 }' Suivi_Clients.csv
N°Facture:
FA-2023-11:
FA-2023-08:
FA-2023-07:
FA-2023-09:
FA-2023-04:
FA-2024-01:
FA-2024-02:
```

2- Afficher les champs n°1 et n°5 du fichier Suivi_Clients.csv sachant que le séparateur entre les champs est ":".

```
esprit@esprit:~/Downloads$ awk -F : '{ print $3,$5}' Suivi_Clients.csv
 Nom  Client  Etat
 AvatarTech  Réglé
 InnovTech  Réglé
 Vivalto Clinic  Réglé
 ClickBank  Réglé
 YorkBank  Réglé
 Idea Hotel Tunisia  Réglé
 Association SOS orphelin  Réglé
```

3- Chercher les lignes du fichier Suivi_Clients.csv dont le mode de paiement est le "chèque"

```
esprit@esprit:~/Downloads$ awk '/chèque/' Suivi_Clients.csv
FA-2023-08: 25/08/2023: InnovTech: 1400Dt: Réglé: 03/12/2023: chèque
FA-2023-07: 15/07/2023: Vivalto Clinic: 3750Dt: Réglé: 21/09/2023: chèque
FA-2024-01: 14/01/2024: Idea Hotel Tunisia: 3880Dt: Réglé: 13/03/2024 chèque
```